构建下一代生成式商业智能（GBI）系统：基于大模型、Agent 架构与实时可视化的深度技术与架构设计报告

1. 执行摘要与引言

企业数据分析领域正处于一场深刻的范式转移之中，即从传统的静态商业智能（BI）向生成式商业智能（Generative Business Intelligence, GBI）演进。传统 BI 依赖于预定义的仪表盘和固定的查询逻辑，往往需要专业的数据分析师介入才能响应新的业务问题。而用户的需求——接入大模型、通过自然语言查询数据库、并实时渲染可视化图表——正是 GBI 系统的核心定义。这种系统旨在消除技术壁垒，使非技术人员能够通过对话直接与企业数据进行交互，从而实现“数据民主化”。

本报告将针对构建这样一个端到端的 Text-to-SQL 及可视化系统进行详尽的技术论证与架构设计。我们将深入探讨支撑该系统的底层核心技术，包括提示工程（Prompt Engineering）、检索增强生成（RAG）、以及大模型在结构化数据推理中的应用；评估当前企业级 Agent 开发框架的适用性；剖析开源生态中的标杆项目；并最终给出一套经过深思熟虑的前后端技术栈设计方案，涵盖从底层数据库交互到前端实时流式渲染的全链路实现细节。

---

2. 大模型领域的关键核心技术

要实现一个能够精准理解自然语言、生成正确 SQL 语句并驱动可视化渲染的系统，仅仅调用一个通用的大语言模型（LLM）是远远不够的。企业级数据库通常包含成百上千张表，且字段命名往往晦涩难懂，业务逻辑复杂。因此，必须引入一系列特定于 Text-to-SQL 领域的增强技术来弥补通用模型的不足。

2.1 提示工程与推理模式（Reasoning Patterns）

在 Text-to-SQL 任务中，Prompt Engineering（提示工程）不仅是输入的艺术，更是引导模型进行逻辑推理的关键机制。

2.1.1 思维链（Chain-of-Thought, CoT）与分解

直接要求模型输出 SQL 往往会导致错误，特别是在涉及多表连接（JOIN）或复杂聚合计算（如“计算上个季度同比增长率”）时。采用思维链技术，强制模型在生成 SQL 之前先生成自然语言的推理步骤，可以显著提高准确率 [1]。例如，系统应引导模型输出如下思考过程：“首先，识别用户查询的时间范围是‘上个季度’；其次，需要连接 orders 表和 customers 表；然后，按 region 字段进行分组统计；最后，编写 SQL 语句。” 这种中间推理步骤不仅有助于模型自我纠错，也为后续的调试提供了可解释性 [2]。

2.1.2 少样本学习（Few-Shot Learning）与动态示例选择

零样本（Zero-Shot）推理在复杂场景下表现不佳。研究表明，在提示词中包含 3-5 个与当前问题相似的“问题-SQL”对（Golden pairs），可以大幅提升模型的 Schema 链接能力 [3]。动态上下文构建：这里的核心技术在于“动态性”。不能硬编码固定的示例，而应构建一个基于向量数据库（Vector DB）的示例库。当用户提问时，系统首先对用户的问题进行语义嵌入（Embedding），然后在向量库中检索最相似的历史问答对，并将其作为上下文注入到 Prompt 中 [4]。这使得模型能够模仿之前的正确写法，学习特定领域的业务逻辑（例如，“高价值客户”的定义可能是 spent > 10000）。

2.2 检索增强生成（Retrieval-Augmented Generation, RAG）在结构化数据中的应用

通用的 RAG 技术主要用于处理非结构化文本（如 PDF 文档），而在 Text-to-SQL 场景下，RAG 的对象变成了数据库的 Schema（元数据）。

2.2.1 Schema Linking（模式链接）

企业数据库可能包含数千个字段，直接将所有 DDL（数据定义语言）放入 Context Window 会超出 Token 限制，且引入大量噪声，导致模型产生幻觉 [6]。核心技术路径：

1. **Schema** 索引：将表名、列名、列描述、甚至列的别名进行向量化索引。
2. 检索与过滤：在生成 SQL 前，先运行一个检索步骤，根据用户问题召回最相关的 Top-N 张表和 Top-K 个字段。
3. **Prompt** 压缩：仅将这些过滤后的 Schema 信息提供给 LLM，从而聚焦模型的注意力，提高生成 SQL 的准确性 [3]。

2.2.2 表格增强生成（Table-Augmented Generation, TAG）

仅知道列名（如 status）往往不足以生成正确的过滤条件，因为模型不知道该列的具体取值（是 0/1，还是 Active/Inactive）。TAG 技术涉及在 RAG 过程中检索列的“样本数据”或“高频去重值” [6]。例如，当用户问“显示所有已发货订单”时，如果 Schema 信息中附带了 status 列的样本值 ``，模型就能准确生成 WHERE status = 'Shipped'，而不是猜测性的 WHERE status = 'delivered' [8]。

2.3 自我纠错与执行反馈循环（Execution-Feedback Loop）

大模型生成的代码（SQL）不仅是文本，更是可执行的逻辑。利用这一特性，可以构建一个“生成-执行-修正”的闭环系统。

1. 语法验证：在执行前，使用 SQL 解析器（如 SQLGlot）检查语法错误。
2. 试运行（**Dry Run**）：在沙箱环境或只读事务中尝试执行 SQL。
3. 错误捕获与重生成：如果数据库返回错误（如“Column not found”），系统将捕获该错误信息，将其作为新的 Prompt 反馈给 LLM，要求其“根据错误信息修正 SQL” [3]。这种 Agentic 的工作流是提升系统鲁棒性的关键，被称为 Self-Correction 或 Self-Repair 机制。

2.4 约束解码与结构化输出（Constrained Decoding）

为了满足“前端实时渲染可视化图表”的需求，LLM 的输出不能仅仅是 SQL 结果，还必须包含用于驱动图表库（如 ECharts 或 Vega-Lite）的配置 JSON。核心技术：利用 OpenAI 的 JSON Mode 或开源模型中的 Grammar-based Sampling（如 Llama.cpp 的 GBNF，或 Outlines 库 ）。这些技术强制模型的输出严格符合预定义的 JSON Schema [10]。这对于前端稳定性至关重要，因为任何 JSON 格式错误都会导致图表渲染失败。

---

3. 企业级 Agent 开发框架选型

构建此类系统需要复杂的编排能力（Orchestration），将意图识别、Schema 检索、SQL 生成、执行、图表生成等步骤串联起来。选择合适的框架可以事半功倍。

3.1 LangChain 与 LangGraph：当前事实标准

LangChain 是目前最广泛采用的 LLM 开发框架，提供了丰富的组件来处理 Prompt 模板、模型调用和工具链 [12]。然而，对于复杂的 Text-to-SQL 流程，**LangGraph**（LangChain 的扩展）是更优的选择 [13]。

- 架构优势：LangGraph 引入了“状态图”（State Graph）的概念，允许开发者将 Agent 的工作流建模为图结构，包含节点（Node，如“生成SQL”）和边（Edge，如“执行成功则跳转到可视化，失败则跳转到修正”）。
- 循环能力：与传统的有向无环图（DAG）不同，LangGraph 原生支持循环（Cycles），这对于实现上述的“自我纠错”循环至关重要。
- 状态管理：它能够持久化 Agent 的状态，使得多轮对话（Multi-turn）和人类介入（Human-in-the-loop）成为可能 [13]。
- 适用性：极度适合需要精细控制执行逻辑、错误恢复和状态保持的复杂企业级应用。

3.2 LlamaIndex：以数据为中心的框架

如果系统的核心挑战在于极其庞大和复杂的数据库 Schema 检索，**LlamaIndex** 可能比 LangChain 更具优势。

- 专注检索（**Retrieval-First**）：LlamaIndex 在数据索引（Indexing）方面表现卓越。它提供了专门的 NLSQLTableQueryEngine 和 SQLTableRetrieverQueryEngine，内置了高效的 Schema 检索策略 [14]。
- 对象索引（**Object Indexing**）：它允许将 SQL 表的元数据作为对象进行索引，支持比纯文本 RAG 更精细的结构化数据检索 [16]。
- 适用性：适合数据源极其复杂、Schema 规模巨大（如数万张表），且对检索精度要求极高的场景。

3.3 框架选型建议

针对本需求（SQL生成 + 实时可视化），建议采用 **LangGraph**。原因在于，本系统的核心难点不仅是检索，还在于生成 SQL 后的执行控制流（Execution Flow）、错误恢复以及后续生成可视化配置 JSON 的复杂逻辑编排。LangGraph 的图导向架构能更清晰地表达这种“规划-执行-观察-修正”的 Agent 模式 [9]。

---

4. 开源项目推荐与参考架构

在从零开发之前，参考或基于成熟的开源项目进行二次开发是明智的选择。以下项目在 Text-to-SQL 领域具有代表性：

4.1 Vanna.ai：生产级 Text-to-SQL Agent

Vanna.ai 是一个基于 Python 的开源框架，专为 Text-to-SQL 任务设计，其架构非常契合本需求。

- 核心特性：
  - **RAG** 原生集成：Vanna 内置了对 DDL、文档和查询历史（SQL/Question Pairs）的训练和检索机制。它不训练模型权重，而是训练“检索库”，这被称为“RAG-based training” [17]。
  - 自我学习（**Tool Memory**）：每次成功的查询都会被记录，随着使用频率增加，系统会越来越懂业务逻辑 [17]。
  - 可视化支持：Vanna 内置了使用 Plotly/ECharts 生成图表的功能，能够根据 SQL 结果自动推荐合适的图表类型 [19]。
  - 前端集成：提供了基于 Flask 的后端 API 和简单的 Streamlit 前端，同时也支持集成到 React 等现代前端中 [20]。
- 推荐理由：Vanna 几乎是一个开箱即用的解决方案，解决了 Prompt 构建、RAG 检索和可视化生成的底层脏活，非常适合快速验证和企业落地。

4.2 DB-GPT：全栈数据智能框架

DB-GPT 是一个更庞大的 AI 原生数据应用开发框架。

- 核心特性：
  - 隐私保护：强调私有化部署，支持在本地运行大模型，确保数据不出域 [21]。
  - **SMMF**（多模型管理）：内置了对各种开源模型（如 Llama 3, Qwen）的管理和调度能力。
  - **AWEL**（智能体编排语言）：拥有一套自己的 Agent 编排语言，专门处理数据流 [21]。
  - **GBI** 支持：提供了生成式 BI 的完整模块，包括报表生成和数据分析。
- 推荐理由：如果企业对数据隐私有极高要求，且需要私有化部署整个 LLM 栈，DB-GPT 是最佳参考。

4.3 WrenAI：基于语义层的 Text-to-SQL

WrenAI 引入了**语义层（Semantic Layer）**的概念，通过建模定义语言（MDL）来管理业务逻辑 [23]。

- 核心理念：不直接让 LLM 面对原始数据库，而是通过一个中间的语义模型（Semantic Model）来交互。这解决了“原始表结构混乱”的问题。
- 推荐理由：适合数据库结构极其复杂、命名极不规范的老旧系统，通过语义层清洗定义后再暴露给 LLM [24]。

---

5. 详细底层架构设计与技术栈建议

基于上述分析，为满足“接入大模型”、“操作数据库”、“实时可视化渲染”三大需求，以下是推荐的端到端架构设计。

5.1 总体架构分层

系统可划分为四层：

1. 交互层（**Interaction Layer**）：前端 UI，负责用户输入、流式响应渲染、图表展示。
2. 编排层（**Orchestration Layer**）：后端 API，负责 Agent 状态管理、RAG 检索、工具调用。
3. 认知层（**Cognitive Layer**）：LLM 服务，负责意图理解、SQL 生成、代码生成。
4. 数据层（**Data Layer**）：业务数据库、向量数据库（存储 Schema 索引）、缓存层。

5.2 前端技术栈设计：React + 实时流式渲染

为了实现极致的用户体验，前端必须支持**流式（Streaming）**交互，即 LLM 生成一个字，前端显示一个字，且图表应该在数据准备好后立即渲染，而不是等待整个回答结束。

- 核心框架：**React (Next.js)**
  - 理由：React 组件化生态丰富，Next.js 提供了优秀的服务器端渲染（SSR）和 API 路由支持，便于与后端集成。
- **AI** 交互库：**Vercel AI SDK**
  - 关键作用：这是一个专为构建 AI 应用设计的库，提供了 useChat 和 useCompletion 等 Hooks。它能自动处理前后端的流式传输（Streaming），使得前端开发者无需手动解析复杂的 SSE（Server-Sent Events）数据流 [25]。
  - 工具调用支持：它支持 LLM 的 Function Calling 结果解析，当后端 Agent 决定“查询数据库”时，前端可以显示“正在查询...”的状态，并在收到 JSON 数据后自动切换为图表组件 [25]。
- 可视化库：**Vega-Lite vs. ECharts**
  - 推荐：**Vega-Lite** [27]
    - 理由：Vega-Lite 采用声明式 **JSON** 语法（Declarative JSON）。这对于 LLM 来说非常友好，LLM 生成一段描述图表的 JSON 比生成一段命令式的 JavaScript 代码（如 ECharts 的 option 对象）要准确得多，且不易出错 [28]。
    - 实现：前端使用 react-vega 组件，直接将 LLM 输出的 JSON 配置和 SQL 查询结果数据绑定即可渲染。
  - 备选：**Apache ECharts**
    - 理由：如果需要极其酷炫、高度定制化的图表（如复杂的 3D 地图），ECharts 是行业

标准。但需要配合更强的 Prompt Engineering 来确保 LLM 生成的配置项符合 ECharts 规范 [29]。

5.3 后端技术栈设计：Python + FastAPI + LangGraph

后端是系统的“大脑”，负责协调 LLM、数据库和向量检索。

- 开发语言：**Python**
  - 理由：AI 和数据领域的通用语言，拥有最完善的 LLM 库（LangChain, LlamaIndex）和数据驱动（Pandas, SQLAlchemy）生态 [31]。
- **Web** 框架：**FastAPI**
  - 理由：高性能，原生支持 Python 的 asyncio。这对于处理 LLM 的流式响应至关重要。FastAPI 可以轻松建立 WebSocket 或 SSE 连接，将 Agent 的中间思考过程（Thinking Process）和最终结果实时推送到前端 [32]。
- **Agent** 编排：**LangGraph**
  - 设计模式：构建一个状态图。
    1. **Node 1: Intent Classification** - 判断用户意图（是闲聊还是查数据）。
    2. **Node 2: Schema Retrieval** - 基于问题检索相关的表结构（RAG）。
    3. **Node 3: SQL Generation** - 调用 LLM 生成 SQL。
    4. **Node 4: Syntax Check & Dry Run** - 验证 SQL。
    5. **Node 5 (Conditional): Error Correction** - 如果出错，回滚到 Node 3 并附带错误信息。
    6. **Node 6: Execution** - 在数据库执行 SQL，获取 Pandas DataFrame。
    7. **Node 7: Visualization Gen** - 将 DataFrame 的元数据（列名、数据类型）传给 LLM，让其推荐图表类型并生成 Vega-Lite JSON 规范。
- 数据库连接：**SQLAlchemy (Async)**
  - 理由：现代 ORM，支持异步查询，避免阻塞主线程。
- 向量数据库：**ChromaDB** 或 **pgvector**
  - 理由：用于存储 Schema 的 Embedding。如果是基于 PostgreSQL 的技术栈，直接使用 pgvector 插件是最轻量级的选择，无需引入新的基础设施 [18]。

5.4 关键实现细节：从自然语言到图表的完整数据流

为了让架构设计更加具象，以下描述一个典型请求的生命周期：

1. 用户输入：“展示上个月各地区的销售额趋势。”
2. **Schema RAG**（后端）：系统将“销售额”、“地区”、“趋势”向量化，在向量库中检索到 orders 表（含 amount, date）和 regions 表（含 name）。
3. **Prompt** 构建（后端）：将检索到的表结构 + 3个类似的少样本示例 + 用户问题组装成 Prompt。
4. **SQL** 生成（**LLM**）：LLM 生成 SELECT region.name, SUM(orders.amount) as total_sales, DATE_TRUNC('month', orders.date)... GROUP BY...。
5. 安全执行（后端）：后端在只读权限下执行 SQL，获取结果集（List of Dicts）。
6. 可视化决策（**LLM**）：后端将 SQL 结果的前几行数据 + 用户问题再次发给 LLM。Prompt：“用户问趋势，数据包含时间和数值，请生成一个折线图（Line Chart）的 Vega-Lite 配置。”

7. 流式响应（前后端）：后端将两部分数据流式推给前端：
    - data: SQL 执行的原始 JSON 数据。
    - viz_spec: Vega-Lite 的 JSON 配置。
8. 前端渲染：React 组件接收到数据，将 data 注入 viz_spec，Vega-Lite 引擎实时绘制出折线图。

---

6. 安全性与性能优化策略

在企业环境中，直接让 LLM 操作数据库存在风险，必须引入严格的管控措施。

6.1 安全架构：沙箱与权限控制

- 只读权限原则：连接数据库的账号必须仅拥有 SELECT 权限，严禁 INSERT, UPDATE, DELETE 或 DROP 权限，从数据库层面物理隔离风险。
- 代码执行沙箱（**Sandbox**）：如果系统不仅仅生成 SQL，还需要生成 Python 代码（例如使用 Pandas 进行复杂分析），必须在隔离环境中运行。推荐使用 **Docker** 容器 或专用的沙箱服务如 **E2B** [33]。E2B 提供了一个安全的云端微虚拟机环境，允许 LLM 生成的代码在其中运行，避免对宿主机造成危害。
- **PII** 敏感信息脱敏：在将数据或 Schema 发送给云端 LLM 之前，必须经过脱敏层。推荐集成 **Microsoft Presidio**，它是一个开源的 PII 识别与匿名化工具，能自动识别并替换姓名、电话、身份证号等敏感信息 [35]。

6.2 性能优化：语义缓存（Semantic Caching）

LLM 调用昂贵且缓慢。

- 策略：引入语义缓存（如 GPTCache 或 Redis VSS）。当用户问“上个月销售额多少？”时，系统计算问题的向量嵌入。如果五分钟前有另一个用户问了“上月卖了多少钱？”，两者的向量高度相似，系统应直接返回缓存中已生成的 SQL 和图表配置，而无需再次调用 LLM [6]。
- 预计算：对于高频的 Dashboard 指标，可以在夜间通过定时任务预先运行 Agent 生成结果，存入缓存，实现毫秒级响应。

6.3 准确率提升：回环学习

- 反馈机制：在前端图表下方添加“赞/踩”按钮。用户反馈的数据（问题+正确的SQL）应自动保存到“Golden Dataset”中。
- 微调（**Fine-Tuning**）：定期使用这些 Golden Data 对开源模型（如 Llama 3 或 CodeLlama）进行微调。研究表明，针对特定 Schema 微调后的 7B 模型，在 Text-to-SQL 准确率上往往能超越通用的 GPT-4 [6]。

---

7. 结论

构建一个支持自然语言查询和实时可视化的 GBI 系统，在技术上已经具备了成熟的可行性。核心在于组合：利用 **LangGraph** 进行复杂的 Agent 逻辑编排，利用 **RAG** 技术解决 Schema 的理解难题，利用 **Vega-Lite** 和 **React** 实现前端的动态渲染，并依托 **FastAPI** 和 **Python** 生态构建健壮的后端。

对于起步阶段，建议优先基于 **Vanna.ai** 开源框架进行原型验证，因为它已经封装了 RAG 和 Visualization 的最佳实践，能大幅缩短开发周期。随着业务复杂度增加，再逐步向自定义的 LangGraph 架构迁移，并引入微调和语义层等高级特性，以满足企业对准确率和安全性的严苛要求。通过这一架构，企业将能够真正解锁数据资产，赋予每一位员工数据分析的能力。

引用的著作

1. LLM text-to-SQL solutions: Top challenges and tips - K2view, 访问时间为 二月 5, 2026，https://www.k2view.com/blog/llm-text-to-sql/
2. Text to SQL: The Ultimate Guide for 2025 | by Ayushg - Medium, 访问时间为 二月 5, 2026，https://medium.com/@ayushgs/text-to-sql-the-ultimate-guide-for-2025-3fa4e78cbdf9
3. Techniques for improving text-to-SQL | Google Cloud Blog, 访问时间为 二月 5, 2026，https://cloud.google.com/blog/products/databases/techniques-for-improving-text-to-sql
4. A Quick review of LLM for Data Visualization - vizGPT, 访问时间为 二月 5, 2026，https://vizgpt.ai/docs/blog/llm-for-viz
5. Build your gen AI–based text-to-SQL application using RAG, powered by Amazon Bedrock (Claude 3 Sonnet and Amazon Titan for embedding) | Artificial Intelligence, 访问时间为 二月 5, 2026，https://aws.amazon.com/blogs/machine-learning/build-your-gen-ai-based-text-to-sql-application-using-rag-powered-by-amazon-bedrock-claude-3-sonnet-and-amazon-titan-for-embedding/
6. LLM & AI Models for Text-to-SQL: Modern Frameworks and Implementation Strategies, 访问时间为 二月 5, 2026，https://promethium.ai/guides/llm-ai-models-text-to-sql/
7. From Natural Language to SQL: Review of LLM-based Text-to-SQL Systems - arXiv, 访问时间为 二月 5, 2026，https://arxiv.org/html/2410.01066v2
8. How we built Text-to-SQL at Pinterest | by Pinterest Engineering - Medium, 访问时间为 二月 5, 2026，https://medium.com/pinterest-engineering/how-we-built-text-to-sql-at-pinterest-30bad30dabff
9. From Queries to Insights: Agentic LLM Pipelines for Spatio-Temporal Text-to-SQL - arXiv, 访问时间为 二月 5, 2026，https://arxiv.org/html/2510.25997v1
10. How can I get LLM to only respond in JSON strings? - Stack Overflow, 访问时间为 二月 5, 2026，https://stackoverflow.com/questions/77407632/how-can-i-get-llm-to-only-respond-in-json-strings

11. What's the BEST local LLM for JSON output, while also being smart? - Reddit, 访问时间为 二月 5, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1ex6ngu/whats_the_best_local_llm_for_json_output_while/
12. Top AI Agent Frameworks in 2025 - Codecademy, 访问时间为 二月 5, 2026, https://www.codecademy.com/article/top-ai-agent-frameworks-in-2025
13. The Best AI Agent Frameworks for 2026 (Ranked by Someone Who's Shipped With All of Them) | HackerNoon, 访问时间为 二月 5, 2026, https://hackernoon.com/the-best-ai-agent-frameworks-for-2026-ranked-by-someone-whos-shipped-with-all-of-them
14. LangChain vs LlamaIndex: A Detailed Comparison - DataCamp, 访问时间为 二月 5, 2026, https://www.datacamp.com/blog/langchain-vs-llamaindex
15. Llamaindex vs Langchain: What's the difference? - IBM, 访问时间为 二月 5, 2026, https://www.ibm.com/think/topics/llamaindex-vs-langchain
16. Top Agentic AI Frameworks in 2025: Which One Fits Your Needs? - Medium, 访问时间为 二月 5, 2026, https://medium.com/data-science-collective/top-agentic-ai-frameworks-in-2025-which-one-fits-your-needs-0eb95dcd7c58
17. Documentation - Vanna AI, 访问时间为 二月 5, 2026, https://vanna.ai/docs
18. Text-to-SQL Just Got Easier: Meet Vanna AI, Your Text-to-SQL Assistant - Medium, 访问时间为 二月 5, 2026, https://medium.com/mitb-for-all/text-to-sql-just-got-easier-meet-vanna-ai-your-rag-powered-sql-sidekick-e781c3ffb2c5
19. Vanna AI, 访问时间为 二月 5, 2026, https://vanna.ai/
20. How to Use Vanna.ai to Query Your Database with Open-Source Language Models, 访问时间为 二月 5, 2026, https://dev.to/aairom/how-to-use-vannaai-to-query-your-database-with-open-source-language-models-5ado
21. DB-GPT: Overview, 访问时间为 二月 5, 2026, http://docs.dbgpt.cn/docs/overview/
22. DB-GPT: Defining the Next Generation of Database Interaction Through Privatized LLM Technology | by Dbgpt | Medium, 访问时间为 二月 5, 2026, https://medium.com/@dbgpt0506/db-gpt-defining-the-next-generation-of-database-interaction-through-privatized-llm-technology-990f85bf769d
23. Why the Semantic Layer is Essential for Reliable Text-to-SQL and How Wren AI Brings it to Life, 访问时间为 二月 5, 2026, https://getwren.ai/post/why-the-semantic-layer-is-essential-for-reliable-text-to-sql-and-how-wren-ai-brings-it-to-life
24. Wren AI: Open-Source Text-to-SQL AI Agents! - YouTube, 访问时间为 二月 5, 2026, https://www.youtube.com/watch?v=g-wuKHlh944
25. Streaming Custom Data - AI SDK UI, 访问时间为 二月 5, 2026, https://ai-sdk.dev/docs/ai-sdk-ui/streaming-data
26. Streaming - Vercel, 访问时间为 二月 5, 2026, https://vercel.com/docs/functions/streaming-functions
27. Vega and Vega-Lite, 访问时间为 二月 5, 2026, https://vega.github.io/

28. The Ultimate Guide to the Vega-Lite MCP Server: Turn Your AI into a Data Visualization Pro, 访问时间为 二月 5, 2026, https://skywork.ai/skypage/en/vega-lite-mcp-server-data-visualization/1980471103827070976
29. Build Stunning Charts using LLM, Natural Language to Echarts | by Karthi Keyan | Medium, 访问时间为 二月 5, 2026, https://medium.com/@karthik429.vk/ai-based-sql-to-stunning-graphs-ai-powered-echarts-generator-806722b10b1f
30. ECharts Options - Evidence Docs, 访问时间为 二月 5, 2026, https://docs.evidence.dev/components/charts/echarts-options
31. Containerized Python Development - Part 1 - Docker, 访问时间为 二月 5, 2026, https://www.docker.com/blog/containerized-python-development-part-1/
32. How to stream LLM response from FastAPI to React? - Stack Overflow, 访问时间为 二月 5, 2026, https://stackoverflow.com/questions/78826168/how-to-stream-llm-response-from-fastapi-to-react
33. 5 Code Sandboxes for Your AI Agents - KDnuggets, 访问时间为 二月 5, 2026, https://www.kdnuggets.com/5-code-sandbox-for-your-ai-agents
34. We built open-source SDK for adding custom code interpreters to AI apps : r/Python - Reddit, 访问时间为 二月 5, 2026, https://www.reddit.com/r/Python/comments/1czr85u/we_built_opensource_sdk_for_adding_custom_code/
35. Microsoft Presidio SVN: Scalable, Structured Sensitive Data Detection for Modern Pipelines, 访问时间为 二月 5, 2026, https://hoop.dev/blog/microsoft-presidio-svn-scalable-structured-sensitive-data-detection-for-modern-pipelines/
36. Presidio by Microsoft: A Practical Guide to Detecting and Masking PII at Scale - Medium, 访问时间为 二月 5, 2026, https://medium.com/@nkbvikram/presidio-by-microsoft-a-practical-guide-to-detecting-and-masking-pii-at-scale-c3b39ce4f52c
37. Generating value from enterprise data: Best practices for Text2SQL and generative AI - AWS, 访问时间为 二月 5, 2026, https://aws.amazon.com/blogs/machine-learning/generating-value-from-enterprise-data-best-practices-for-text2sql-and-generative-ai/